LÖSUNGEN · LEHREREXEMPLAR

W0·3 – Die Vorratskammer: Lösungen

Zum Arbeitsblatt `AQ_W0_AB3_Vorratskammer_Reihungen.docx` · IDE-Level 0.3: siehe `AQ_W0_Akademie_LSG.txt` (Klasse Vorrat) · alle Werte mit Java (javac 21) nachgerechnet

Aufgabe 0 – Blitz-Warm-up

Ausdruck	`r.length`	`r[r.length - 1]`	`r[r[4]]`	`r[2] + r[3]`	`r[5]`
Wert	5	1	3 (r[1])	5	Fehler (Index 5 bei Länge 5)

Ausdruck	`s.length()`	`s.charAt(2)`	`s.substring(1, 4)`	`s.charAt(s.length() - 1)`	`(int) s.charAt(0)`
Wert	6	'r'	"orr"	't'	86

`substring(1, 4)` liefert die Zeichen an den Indizes 1, 2, 3 (Ende ausschließlich).

Aufgabe 1 – Regalbau (7 BE)

Zeitpunkt	Index 0	Index 1	Index 2	Index 3	Index 4	Index 5	Bemerkung
nach Zeile 1	0	0	0	0	0	0	
nach Zeile 2	5	0	0	0	0	0	
nach Zeile 3	5	0	0	0	0	2	
nach Zeile 4	5	0	7	0	0	2	
i = 1	5	10	7	0	0	2	
i = 3	5	10	7	14	0	2	
i = 5	5	10	7	14	0	0	überschreibt die 2!

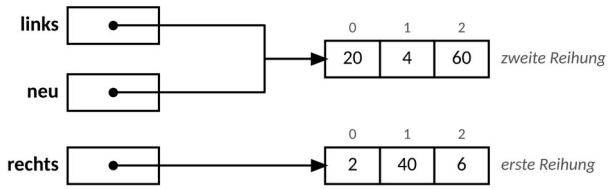
Grün: in diesem Schritt geänderte Fächer. Falle: Im Durchlauf i = 5 wird `regal[5] = regal[4] * 2 = 0` – die 2 aus Zeile 3 wird überschrieben. Endzustand **{5, 10, 7, 14, 0, 0}** (4 BE).

b) Die Reihung hat die Länge 6, gültig sind die Indizes 0 bis 5. Index 6 existiert nicht → ArrayIndexOutOfBoundsException (1 BE). **c)** In `namen[0]` steht **null** (Standardwert für Zeichenketten). `null.length()` ist ein Methodenaufruf ohne Objekt → **NullPointerException**, Abbruch (2 BE).

Aufgabe 2 – for oder for-each? (5 BE)

Nr.	Schleife	Schleifenkopf	Begründung
1	for-each	`for (int x : vorrat)`	nur lesen und addieren (for geht auch)
2	for	`for (int i = 0; i < vorrat.length; i++)`	`vorrat[i] = vorrat[i] + 1` braucht den Index; for-each änderte nur die Kopie x
3	for	`for (int i = 0; i < vorrat.length; i++)`	der Index ist das Ergebnis
4	for-each	`for (int x : vorrat)`	nur lesen und vergleichen (for geht auch)
5	for	`for (int i = 0; i < vorrat.length - 1; i++)`	Vergleich `vorrat[i] > vorrat[i + 1]`; Grenze length − 1, sonst Indexfehler (alternativ i von 1 bis length − 1 mit `vorrat[i - 1] > vorrat[i]`)

Aufgabe 3 – Das Regal-Rätsel (8 BE)



a) Speicherbild (3 BE): zwei Reihungen; rechts zeigt auf die erste (dort hat Zeile 4 die 40 eingetragen), links und neu zeigen nach Zeile 6 beide auf die zweite. Nach Zeile 2 zeigte links noch auf die erste Reihung – Zeile 6 „verbiegt" nur den Pfeil von links.

b) links: **{20, 4, 60}** · rechts: **{2, 40, 6}** · neu: **{20, 4, 60}** (2 BE).

c) `links == neu` → **true** (gleiche Referenz), `rechts == neu` → **false**. `==` vergleicht bei Reihungen die Referenzen; zwei verschiedene Reihungen mit gleichen Werten sind verschiedene Objekte (mit Java geprüft: `false`). Inhaltsvergleich: gleiche Länge prüfen und dann in einer Schleife jedes Paar `a[i] != b[i]` prüfen (vgl. Methode D in Aufgabe 7) (3 BE).

Aufgabe 4 – Ein Fach weiterrücken (10 BE)

i	(i + 1) % r.length	r[i]	neu (nach dem Durchlauf)
0	1	7	{0, 7, 0}
1	2	1	{0, 7, 1}
2	0	5	{5, 7, 1}

Rückgabe **{5, 7, 1}** (3 BE). **b)** Die Methode liest r nur (`r[i]` rechts vom `=`) und schreibt ausschließlich in die neu erzeugte Reihung. Mit `int[] neu = r;` verweisen neu und r auf dieselbe Reihung: Die Methode überschreibt Werte, die sie später noch lesen muss, und verändert das Original – für {7, 1, 5} entsteht **{7, 7, 7}** (mit Java geprüft) (2 BE).

c) Bei Länge 0 ist die Schleifenbedingung `0 < 0` sofort falsch; der Rumpf und damit `% r.length` werden nie ausgewertet. `new int[0]` ist erlaubt und wird zurückgegeben (Ergebnis {}) (2 BE). **d)** Struktogramm (3 BE):

verschoben(r: Reihung von Ganzzahl): Reihung von Ganzzahl
n ← Länge von r
neu ← neue Reihung von Ganzzahl der Länge n
für i von 0 bis n − 1
neu[(i + 1) % n] ← r[i]
gib neu zurück

Aufgabe 5 – Die verbotenen Etiketten (8 BE)

```
public static boolean endetMit(String s, String ende) {
    if (ende.length() > s.length()) {
        return false;
    }
    return s.substring(s.length() - ende.length(), s.length()).equals(ende);
}
public static int anzahlLeerzeichen(String s) {
    int anzahl = 0;
    for (int i = 0; i < s.length(); i++) {
        if (s.charAt(i) == ' ') {
            anzahl++;
        }
    }
    return anzahl;
}
public static boolean istGrossbuchstabe(char c) {
    return c >= 'A' && c <= 'Z';     // gleichwertig: (int) c >= 65 && (int) c <= 90
}
```

a) 5 BE: endetMit mit Längenabfrage (1) und substring/equals oder Zeichenvergleich von hinten (1); anzahlLeerzeichen mit Schleife und Zeichenvergleich `' '` (2); istGrossbuchstabe über den Bereich 'A' … 'Z' (1). Geprüft: endetMit("Feuerball", "ball") = true, endetMit("Ball", "Feuerball") = false; anzahlLeerzeichen("Der rote Drache") = 3.

Ausdruck	Ergebnis	Begründung
`"Drache".compareTo("Drachen")`	negativ (-1)	Drache ist ein Anfangsstück von Drachen, also kürzer → davor
`"elf".compareTo("Elf")`	positiv (32)	'e' (101) kommt nach 'E' (69)
`"Eis".compareTo("Feuer")`	negativ (-1)	'E' vor 'F'
`"Rune".compareTo("Rune")`	0 (0)	gleich

b) 2 BE (das Vorzeichen genügt). **c)** `(int) 'a'` = **97** · `(char) ('a' + 2)` = **'c'** · `(char) ((int) 'D' + 32)` = **'d'** (1 BE).

Aufgabe 6 – Level 0.3

Musterlösung und häufige Fehler: `AQ_W0_Akademie_LSG.txt`, Klasse Vorrat. **b)** Referenz-Rätsel: `a[0] = 99` (a und b verweisen auf dieselbe Reihung). Nach verdopple: **{8, 0, 14, 0, 24, 18}** – verdopple verändert die Reihung des Hauptprogramms (Vorgriff auf DS 5: Referenzübergabe).

Aufgabe 7 – Bug-Hunt (11 BE)

Methode	Zeile	Fehler	Korrektur
A	2	Off-by-one: für i = r.length − 1 greift `r[i + 1]` hinter das Ende	`for (int i = 0; i < r.length - 1; i++) {`
B	2	`neu = r` kopiert nur die Referenz → r wird verändert	`int[] neu = new int[r.length];`
C	4	`"e"` ist eine Zeichenkette; Zeichen und Zeichenkette sind nicht vergleichbar (Compilerfehler)	`s.charAt(i) == 'e' || s.charAt(i) == 'E'`
D	2	`==` vergleicht die Referenzen, nicht die Inhalte	siehe d)

b) z. B. {3, 1, 2} → false (die Methode endet vor dem Ende der Reihung); {1, 2, 3} → Abbruch mit ArrayIndexOutOfBoundsException. Abbruch bei **jeder** aufsteigend sortierten Reihung mit mindestens einem Element (auch {5}); die leere Reihung liefert true (2 BE). **c)** Ausgabe `3 3` – b wurde mitverändert (1 BE).

d) Korrekturen A–C siehe Tabelle (je 1 BE); Methode D (1 BE):

```
public static boolean gleicherInhalt(int[] a, int[] b) {
    if (a.length != b.length) {
        return false;
    }
    for (int i = 0; i < a.length; i++) {
        if (a[i] != b[i]) {
            return false;
        }
    }
    return true;
}
```

Für Schnelle – Die Lotterie der Kammer

6 aus 49: mit `Vorrat.enthaelt` vor dem Eintragen prüfen; eine Zahl nur übernehmen, wenn sie noch nicht in den bisher belegten Fächern steht (Achtung: Die noch leeren Fächer enthalten 0 – das stört hier nicht, weil 0 nie gezogen wird). **Reindrängler:** eingefuegt({4, 8, 15}, 1, 99) = {4, 99, 8, 15}, an Position 0: {99, 4, 8, 15}, an Position 3: {4, 8, 15, 99}. **Caesar:** "xyz" mit k = 3 → "abc"; "kdoor zhow" mit k = −3 → "hallo welt".

```
public static int[] eingefuegt(int[] r, int pos, int wert) {
    int[] neu = new int[r.length + 1];
    for (int i = 0; i < pos; i++) {
        neu[i] = r[i];
    }
    neu[pos] = wert;
    for (int i = pos; i < r.length; i++) {
        neu[i + 1] = r[i];
    }
    return neu;
}
```